SHERLOCK

Security Review For Centrifuge

Collaborative Audit Prepared For: **Centrifuge**
Lead Security Expert(s): **0x52**
0xeix
Date Audited: **May 12 - May 14, 2026**

Introduction

A passthrough vault is an immutable, non-custodial contract that lets many investors deposit into and redeem from a Centrifuge vault. Requests are forwarded directly to the underlying vault and settled in order. Investors receive the actual share token and the passthrough vault never holds funds on their behalf. Because the passthrough vault is the sole participant in the underlying, it can enforce its own investor permissions independently of the underlying vault's permissions.

Scope

Repository: centrifuge/passthrough-vaults

Audited Commit: 31773681588f2a1fb2ca51dcb84ace134ac8c67f

Final Commit: bbe0c9bd495901cbb180560b06b3c4eff64b5855

Files:

- src/libraries/QueueLib.sol
- src/PassthroughVault.sol

Final Commit Hash

bbe0c9bd495901cbb180560b06b3c4eff64b5855

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	4

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: The vault contract fails to fully comply with EIP7714 [RESOLVED]

Source: https://github.com/sherlock-audit/2026-05-centrifuge-update-may-12th-2026/issues/3

Summary

`PassthroughVault` implements the ERC-7714 permission check function `isPermissioned(address)` but does not implement ERC-165 `supportsInterface(bytes4)`, making ERC-7714 support undiscoverable by standards-compliant integrations.

Vulnerability Detail

ERC-7714 requires contracts to support ERC-165 and return true for interface ID `0x78d77ecb`. However, `PassthroughVault` only implements `isPermissioned(address)`. An integration checking support with:

```
vault.supportsInterface(0x78d77ecb)
```

would revert because `supportsInterface(bytes4)` does not exist. The imported `IERC7714` from `protocol/misc/interfaces/IERC7540.sol` also only declares `isPermissioned(address)` and does not inherit ERC-165.

Impact

Integrations relying on ERC-165 discovery will fail to detect ERC-7714 support, even though the contract functionally performs permission checks.

Tool Used

Manual Review

Recommendation

Consider adding ERC-165 support to `PassthroughVault`:

```
+function supportsInterface(bytes4 interfaceId) external pure returns (bool) {
+    return interfaceId == 0x01ffc9a7 || interfaceId == 0x78d77ecb;
+}
```

for the contract to be fully compliant with this EIP requirement:

```
Smart contracts implementing this Vault standard MUST implement the ERC-165
↪  supportsInterface function.
Contracts MUST return the constant value true if 0x78d77ecb is passed through the
↪  interfaceID argument.
```

Discussion

hieronx

We are not implementing `supportsInterface` for ERC-7575 and ERC-7540 either. Agree we should add to the natspec comments that it is not fully ERC-7741 compatible.

hieronx

On the severity: I would say this is low/info. The contract is not implementing `supportsInterface` for other ERCs either, so this is clearly a documentation issue.

rodiontr

> On the severity: I would say this is low/info. The contract is not implementing `supportsInterface` for other ERCs either, so this is clearly a documentation issue.

agreed and already downgraded, thank you

Issue L-2: Forced partial claims can cause rounding loss [RESOLVED]

Source: https://github.com/sherlock-audit/2026-05-centrifuge-update-may-12th-2026/issues/4

Summary

When `claimForAll` is enabled, anyone can force a controller to claim partially by calling `deposit()` or `redeem()` with `receiver == controller`. Because the underlying rounds claim outputs down while consuming rounded-up settlement capacity, forced tiny claims can cause value loss.

Vulnerability Detail

`PassthroughVault` allows third-party claims when:

https://github.com/sherlock-audit/2026-05-centrifuge-update-may-12th-2026/blob/main/passthrough-vaults/src/PassthroughVault.sol#L201

```
controller == msg.sender || claimForAll && controller == receiver
```

An attacker can call with tiny amounts (like 0.6 base units) - for example, assume we have the following math:

```
asset = USDC, 6 decimals
shares = 18 decimals
price = 1 share = 1 USDC
```

The underlying math rounds the amount down. So each call reduces claimable settlement capacity by, let's say, 1 USDC base unit but transfers 0 assets to the victim.

Impact

A third party can grief users by forcing inefficient partial claims, causing rounding loss. The attack is not directly profitable but can destroy small amounts of user value repeatedly when `claimForAll == true`.

Tool Used

Manual Review

Recommendation

When `msg.sender != controller`, only allow full claims:

```
require(shares == type(uint256).max || actualShares == claimable,
↪  InvalidController());
```

Issue L-3: `maxDeposit()` and `maxRedeem()` do not reflect pass-through membership [RESOLVED]

Source: https://github.com/sherlock-audit/2026-05-centrifuge-update-may-12th-2026/issues/5

Summary

`PassthroughVault.maxDeposit()` and `PassthroughVault.maxRedeem()` can return non-zero values for a controller that is no longer `permissioned` by the configured `memberlist`. The write paths enforce `permissioned(controller)`, but the view functions do not apply the same check. This can cause users to see claimable/depositable capacity that cannot actually be used until the controller is re-permissioned.

Code Snippet

PassthroughVault.sol#L136-L142

PassthroughVault.sol#L222-L224

Tool Used

Manual Review

Recommendation

Return `0` from `maxDeposit()` and `maxRedeem()` when `!isPermissioned(controller)`, matching the corresponding mutating function behavior.

Issue L-4: Factory does not validate `asyncDeposit` against the underlying vault kind [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-05-centrifuge-update-may-12th-2026/issues/6

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`PassthroughVaultFactory.newVault()` accepts an arbitrary `asyncDeposit` flag and deploys the pass-through vault without checking that the flag matches the underlying vault type. If `asyncDeposit` is set incorrectly, the deployed vault will expose the wrong deposit flow and only fail later when users interact with unsupported underlying functions. Since protocol-internal vaults expose `vaultKind()`, the factory can validate this at deployment time.

Code Snippet

PassthroughVault.sol#L274-L280

Tool Used

Manual Review

Recommendation

Validate the flag during deployment, for example by requiring `asyncDeposit == (IVault(vault).vaultKind() == VaultKind.Async)`, and revert on mismatches.

Discussion

hieronx

Acknowledged. This is deployer error. We don't think verifying this on constructor is worth the additional code.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.